

Evermon

one-liner

Evermon reads a shipping-documentation inbox, matches SI against draft BL by meaning (not header text), flags the seven fields that actually matter, and escalates to a human with a stated reason when it can't decide.

problem

Clerks manually cross-check SI vs BL — shipper, consignee, notify party, load/discharge port, container count, gross weight — across documents that never use the same labels (“Port of Loading” vs “Load Port”, units hidden in a mixed-language gloss). Slow, and a missed mismatch puts the wrong consignee on a negotiable document. This is real and it's the honest problem — don't inflate it further.

why this is not a wrapper / chatbot / dashboard

The alias-table field alignment (`dock/fields.py`) and the four-reason escalation precedence (`dock/review.py`) are the product — a judge calling GPT with “compare these two documents” gets inconsistent field matching and no auditable escalation policy. **But be honest:** if extraction and comparison are just LLM calls with a prompt, this is a thin wrapper with good scaffolding around it. The pitch only survives if you can show the alias table and comparison logic are deterministic/rule-based (not just prompted), and that escalation reasons are fixed, not model-improvised. If any of that is LLM-guessed under the hood, say so on stage before a judge finds it — don't let them discover it.

market - who pays

Freight forwarders and shipping-line back offices — the buyer is the ops manager who owns error/rework cost and demurrage exposure, not the individual clerk. This is a real B2B wedge but it's crowded (document-matching-for-logistics is a known SaaS category); you need one sharp number — clerk-hours saved per 100 emails, or mismatch-catch rate vs. manual — or this reads as “another OCR-plus-rules tool.”

open-source strategy

Ship the classification rules, alias table, and escalation-reason logic open (dock/classify.py, dock/fields.py, dock/review.py) — that’s the trust argument: a clerk-facing tool that flags negotiable-document errors needs to be auditable, not a black box. Keep document parsing robustness (edge-case PDF/XLSX handling) and any fine-tuned extraction model closed, since that’s the maintenance moat, not the IP moat. Don’t oversell “open source” as a differentiator here — judges will ask what’s actually proprietary, and “the loader” isn’t a good answer.

demo script (3 bullet steps, under 2 minutes)

1. Drop three real emails into the inbox (a spam/broadcast, an invoice chase, an SI+draft-BL pair) — show classify.py sorting them correctly in one pass.
2. Run the SI/BL pair with a deliberately relabeled field (“Load Port” vs “Port of Loading”, mixed-language weight unit) — show it aligns and flags a real mismatch, not a label mismatch.
3. Run a case with a missing/unreadable attachment — show it escalates to NEEDS_REVIEW with the specific stated reason, not a guess.

Cut before the pitch: any claim of “AI-powered” without naming what’s deterministic vs. model-driven — that’s the first thing a technical judge will probe, and a vague answer kills the credibility this pitch depends on.